Adelaide Competitive Programming Club

Round 1 Workshop

What is Comp Prog? + Intro to Python

What is Competitive Programming in AUCPL Div B?

What is Competitive Programming?

- Solving algorithmic and mathematical problems in a gamified contest environment.
- Time pressure, teamwork, leaderboards, and prizes!
- Hopefully a fun and social activity
- Learn new things and improve your hireability!

What is AUCPL Div B?

- We aim to teach a curriculum of competitive programming through these workshops and our contests
- The content will build up through the year. Hopefully you will be a master competitive programmer by the end, ready to take over Div A next year!

This means Div B is totally accessible to beginners! Heres some specifics:

- Contests are done individually
- Each contest is 2 hours
- There are 7 workshops and 6 contests (+AllUni) during the year
- Each contest has 6 problems
- No cash prizes, so we can help you during the contests!

It is meant to lead into Div A, which is a more serious invigilated environment with teams of 3 and cash prizes!

Choosing a Programming Language

- Our platform supports C, C++, Python, and Java. They all have their pros and cons generally, and specifically for competitive programming.
- Generally, use the one you are most confident in.
- C: This is a more limited form of C++, use it only if you only know it.
- C++: This is our generally preferred language, but it is pretty difficult so we won't be teaching it.
- Python: Has some limitations compared to C++ but it is more approachable. We are using it!
- Java: Some university programs focus on Java, but I would always prefer Python or C++.

In Div B, we will primarily teach in Python. We will highlight when we encounter limitations, and encourage you to practice C++ in your own time if you want to reach the next level of competitive programming!

Enough talk! Show me some code!

Variables and Types

Note: You can use https://ide.usaco.guide to run your Python code!

In Python, you can use `=` to create a variable - think of it as storing some value in a named box.

Every variable roughly has a type - some types are

- `int` - integer numbers like `1`, `0`, `-100`
- `str` - strings are text (like a group of characters): `"Hello!"`, `"123  this is a string!"`
- `float` - decimal numbers like `1.234`, `1234.1`, etc
- `bool` - boolean values are like yes/no answers. The only possibilities are `True` and `False`.

```python
myVar = "hello"
myInt = 10
myFloat = -1.0
shouldYouComeToAUCPL = True
```

There are limits on what variables can be called! For instance, they can't start with a number, and they can't be one of the language-reserved keywords like `and`, `for`, `return`, etc. Look it up to learn more!

Variables and Operators

Now we can store some constant values in variables, what about manipulating them? Operators can be used to combine different values, with different results depending on the type! The type inputs and outputs are roughly what you would expect!

Note that anything after `#` is a comment in Python and is ignored by the interpreter - it just gives information to us humans.

```python
var1 = 1 + 2 # 3: int + int -> int
var2 = 1 / 2 # 0.5: int / int -> float
var3 = 1 == 2 # False: Are 1 and 2 the same? No they aren't
var4 = "Hello" + " " + "World" # "Hello World". We can chain these operators
x = 1
x += 1 # Sets x to be x + 1, which is 2 now
```

There are many operators! Sometimes, some type combinations don't make sense. For instance `"Hello" + 5` will give you an error.

Variables and Operators

Some more miscellaneous operators

```python
boolA = True
boolB = False
boolA and boolB # False. Only true if both A and B are True
boolA or boolB # True
8 // 6 # 1. This is integer division, discarding remainder
8 % 6 # 2. This is the remainder, i.e. 8 = 1*6 + 2
8 > 2 # True. 8 is greater than 2. Also >=, <, <=, !=, ==
"Hello" * 3 # "HelloHelloHello"
10**3 # 100. 10 to the power of 3
```

Variables and Operators

You can also convert between types, called casting!

```python
a = int("123") # 123
b = bool(0) # False
c = int(3.5) # 3. This will round towards 0
d = float(3) # 3.0
c = float("5.5") # 5.5
e = bool("False") # True. Careful! This is only False if the string is empty:
f = bool("") # False
```

This will error if your value doesn't make sense.

```python
notAnInt = int("This is clearly not an integer")
# ValueError raised! Program crashed
```

Input and Output: I/O

Input and Output

Now we have some variables, and we can manipulate them. In competitive programming, we can't just hard code values. We need our program to be able to read in inputs, and print out outputs. There are two functions that can help with that.

- `input()` will read a single line in as a string
- `print(x)` prints out what is inside the brackets. They can usually be any type!

```python
print(1,2,3)
# 1 2 3
# Passing multiple arguments to print space-separates them. We can customise this behaviour:
print(1,2,3,sep="\n") # "\n" means newline
# 1
# 2
# 3
myString = input() # Store a line in myString
```

Input and Output

Now, note this input only gives us a string. Often we need to convert this into something more usable!

For instance, say we want to read a number as input and output its double.

```python
myValue = input()
print(myValue + myValue)
```

If the user inputs `1`, program will output `11`. This is because it reads the input as string `"1"`, and `"1"+"1"` is `"11"`.

```python
myValue = int(input())
print(myValue + myValue)
```

Now this executes correctly, and errors if the input contains something other than an int.

Input and Output

Another common case is a line containing space-separated integers. We can read in a list of integers as follows:

```python
myValues = list(map(int,input().split()))
```

We haven't learnt about lists yet, but here is a high-level overview of what is happening:

- `input().split()`: Takes the line as a string and splits it at spaces: `"1 2 3"` turns into `["1","2","3"]`
- This list contains strings still, which isn't what we want. `map(int, ...)` effectively produces a new list by calling `int` on each element of the inputted list.
- `map` doesn't exactly produce a list, it produces a different special object for some more complicated reasons, so we can convert it to a `list` by casting it.

Again, this may be very confusing! We will teach you what lists are soon! Don't worry. However, the above line is a very common pattern, so you will see it in our solutions.

Input and Output

Sometimes we want to print out things that involve variables, you can do this using string concatenation but it isn't fun. There is a special type of string called an f-string which we can use.

```python
name = input()
print(f"Hello, {name}")
```

Notice the f before the first quote. Anything between curly braces in this string will be evaluated as Python code, so we can use it to fill in variables.

There are lots of other powerful things we can do with this to format values. One thing I'll highlight is rounding floats, say we want to print some value to 3 decimal places.

```python
a = 1.23456
print(f"{a:.3f}") # prints 1.235
```

This is a bit confusing, basically `:` specifies that we are going into the formatting section, `.3f` means we are printing a float to 3 decimal places. There are other uses you can look up.

Lists, control flow, functions

Lists

A list is an ordered collection of other values. Previously, our variables only stored a single value, but with lists we can store a variable amount of them!

As can be seen, we can construct lists using square brackets. Then, you can ‘index into’ them to retrieve any particular element (starting from zero, of course)

```python
myList = ["Hello", 1, False, "this is a list"]
print(myList[0]) # prints "Hello"
print(myList[4]) # Errors! There aren't enough elements in the list
myList.append(6.7) # We can add stuff like this
print(myList[4]) # prints 6.7
myList.pop() # This will remove the last element, we can also provide an index
print(myList) # ["Hello", 1, False, "this is a list"]
```

Lists

We can also create lists by ‘slicing’ other lists:

```python
a = [1,2,3,4]
print(a[0:3]) # [1,2,3] - note the right bound is exclusive. This is same as a[:3].
print(a[3:]) # [4]. If we omit the right bound it just takes to the end.
print(a[::2]) # [1,3]. Same as a[0:5:2]. The third number means 'step size'
print(a[-1]) # 4. We can use negative indices to get from the end
print(a[::-1]) # [4,3,2,1] step size of -1 means we are looping through backwards
```

Lists

Some other things we can do to lists

```python
a = [4,2,1,3]
print(len(a)) # 4. amount of elements in the list
a.clear() # a is now []
a = [4,2,1,3]
a.reverse() # a is now [3,1,2,4]
a.sort() # a is now [1,2,3,4]. We can sort descending order using a.sort(reversed=True)
a.count(2) # 1. How many times does 2 appear in the list?

[1,2] + [3,4] # [1,2,3,4]. Add one list to the end of another
```

Sorting will fail if we are mixing datatypes, i.e. you cannot sort [1,“Hello”].

Control flow: Conditionals

We can execute code only if some condition holds true. This is where those boolean types come in handy.

```python
myInt = int(input())
if myInt == 0:
  print("The user inputted 0!")
elif myInt == 1: # This is like `else if`
  print("The user inputted 1!")
else:
  print("The user inputted something else...")
```

You can chain many elifs, and you can use types other than booleans for it. You can also do if statements inline. This prints nonempty, but it would print empty if the list was `[]`.

```python
print("Nonempty" if [1,2,3]) else "empty")
```

Control flow: While loops

Sometimes we might want to repeat some code until a certain condition is true. While loops!

```python
n = 0
while n < 100:
  print("Come to AUCPL!")
  n += 1
```

Each loop, `n` will increase by `1` until the loop stops. Thus, the loop will print the message 100 times.

```python
a = [1,2,3,4]
i = 0
while i < len(a):
  print(a[i])
  i += 1
```

Can you figure out what this code does?

Control flow: For loops

Sometimes we want to execute some code for every element in a container, such as a list.

```python
a = [1,2,3,4]
sum = 0
for x in a:
  sum += x
print(x)
```

What will this print? There is also a useful `range` function we can try.

```python
a = [1,2,3,4]
for i in range(len(a)):
  print(a[i])
```

`range(n)` gives `0,1,...,n-1` (exclusive). You can also do `range(a,b)` to get `a,a+1,...,b-1`.

Functions!

We have actually seen lots of functions already! `print`, `range`, `input`, `int`, `len` are all functions. They take some inputs, and give us some outputs. We can define them as follows:

```python
def addTwoNumbers(a, b):
  sum = a + b
  return sum

print(a + b)
```

`addTwoNumbers` is the name of the function. In the parentheses is the 'parameters', there can be any number of these (or none, as we used `input()`). `return` is a keyword that basically gives back the value passed to it to whoever called the functions.

Functions!

Sometimes we want to make a function without a name, so we can just use it in one spot. There is a lambda keyword for this. For instance `lambda x: x + 1` is a function that takes an integer and returns it plus 1.

This is useful in tandem with `map` as shown before, or other functions like `filter`.

```python
a = [1,2,3,4]
b = list(map(lambda x: x + 1), a)
print(b) # [2,3,4,5]
c = list(filter(lambda x: x % 2 == 0), b)
print(c) # [2,4]
```

Our second lambda is checking if `x % 2 == 0`, i.e. if `x` is even. `filter` removes any elements that return false when given to the function passed.

Dictionaries

We will go into more depth on these later, but basically they are a way you can give a ‘key’ a ‘value’ and look up or modify the value in the future, similar to how words have definitions in a dictionary.

```python
emptyDict = {} # We use curly braces to declare them
myDict = {"a": 1, "b": 2, 5: "hello"}
print(myDict["b"]) # prints 2
print(myDict[2]) # Error! No key for 2
myDict[2] = [1,2,3] # Can store whatever you want in the dict
print(myDict[2]) # [1,2,3]
del myDict["a"] # Remove a key
for key, value in myDict.items():
  print(value)
  # prints 2, "hello", [1,2,3]
```

Standard Library

As you might know, some functions are built into the language and don't need to be defined by you. Some examples are `input` and `print`, as well as `range`, etc. These are made available by default, i.e. you can just use them. There are some other standard library functions that have to be 'imported' to be used. You can read about these in the Python documentation.

```python
import math
print(math.sin(math.pi * 3/4)) # 0.707...
print(math.sqrt(15)) # 3.87... Same as 15 ** 0.5
import statistics
print(statistics.mean([1,3,5,7])) # 4
print(statistics.median([8, 1, -51, 7])) # -8.75
```

There are lots more! Some useful ones are `itertools`, `functools`, `collections`, etc.

Putting it all together!

Using Notes

We have made some examples problems for you to try, but before we get to that we had some general tips for you.

Firstly, all competitive programming contests generally allow paper notes. This means you don't need to memorise everything! For instance, you could write down that snipped that takes a line of `int`s as input. As we progress through the curriculum, there will be more and more useful algorithms that may be hard to memorise but using them would give you an edge. We encourage you to use notes! Write your own and print them out, or use pre-existing notes.

Here are some more tips!

- Use your time effectively! This means not getting stuck on one problem. Maybe you could look through all the problems in the to solve first.
- Test your code before you upload it! There is a penalty for incorrect submission. Even if you are pretty sure it works its beneficial to think about edge cases and write some tests/atleast test the examples.

Putting it all together! Example problem.

We now know somewhat how to code Python. How do we actually competitively program? We have our own website https://aucpl.com/ that hosts the problem statements, describing the problem, how the input looks, how the output should look, and some example cases.

The website also allows you to submit your code for grading, which is how the leaderboard works. This will run your program against a bunch of our own test cases to ensure it produces the correct output.

We have made a problem for you to try!

https://aucpl.com/problem/clockingchaos

Example Solution

Did you manage to solve Clocking Chaos? Heres our solution anyway...

```python
N = int(input())
clocks = list(map(int, input().split()))
voltages = list(map(float, input().split()))
max_speed = 0
for i in range(N):
    speed = clocks[i] * voltages[i]
    if speed > max_speed:
        max_speed = speed

print(f"{max_speed:.3f}")
```

Hopefully one of our handsome presenters can now explain what this code does.

Another example solution

We can do better! At least in having less lines of code, which isn't always better (but it's fun).

```python
n = int(input())


speeds = map(int, input().split())
voltages = map(float, input().split())


best_speed = max(map(lambda x: x[0] * x[1], zip(speeds, voltages)))
print(f"{best_speed:.3f}")
```

This uses some functions you may be unfamiliar with. `zip` takes two lists and 'zips' them up together into a single list, where each element is a pair of two elements. Max just returns the element which is the largest.

Another example problem:

https://aucpl.com/problem/skyline

There are two clear ways to solve this problem, and one is faster than the other. Here, both will be fast enough to be accepted, but in the future thinking about the performance of algorithms is important.

We will go through both after you've had some time to attempt the problem yourselves

Slower Example Solution

```python
n = int(input())
heights = list(map(int, input().split()))
min_diff = 10**9
b1, b2 = 0, 0

for i in range(n - 1):
    for j in range(i + 1, n):
        diff = abs(heights[i] - heights[j])
        if diff < min_diff:
            min_diff = diff
            b1 = min(heights[i], heights[j])
            b2 = max(heights[i], heights[j])

print(b1, b2)
```

Optimal Solution

`sorted` is like `.sort()` on a list, except that it produces a new list rather than sorting one in place (and it is easier to chain like this). Also `None` is a type that only has one value, and is sometimes useful.

Python

```python
n = int(input())
nums = sorted(map(int, input().split()))

minDist = None
bestA, bestB = None, None
for i in range(n-1):
    if minDist is None or nums[i+1]-nums[i] < minDist:
        minDist = nums[i+1] - nums[i]
        bestA = nums[i]
        bestB = nums[i+1]

print(bestA, bestB)
```

Extra resources

Education material

- https://usaco.guide/ has a curriculum all the way from beginner up to expert
- https://cp-algorithms.com/ is more of an assorted algorithms/concepts reference

Contests. We encourage you to compete to practice your skills!

- https://atcoder.jp/ has the best timings, and their beginner contests are quite nice.
- https://codeforces.com/ is the most famous contest platform, but the contests generally start at 1am.
- Leetcode hosts biweekly contests

In the ACPC Discord you can usually find people doing the ABC (ATCoder Beginner Contest) on Saturday nights.